

✿ MERN Stack Internship — Task 5

✦ Task Description

You've built authentication (Task-4). Now evolve your app into a **multi-user Notes service** where **each user manages their own notes**, and **admins** can moderate. Add **server-side validation, search, and pagination**, plus **"owner only" authorization** on update/delete.

What You'll Build (Requirements)

Backend (Express + MongoDB):

1. **Models:** `User`, `Note { title, content, tags[], owner, createdAt }`.
2. **AuthZ (Authorization):**
 - Users: can CRUD only their own notes.
 - Admins: can list/delete any note.
3. **Validation:** Use a schema validator (e.g., **Joi** or built-in checks):
 - title: required, 3–120 chars; content: required; tags: array of strings (optional).
4. **Search & Pagination:**
 - `/api/notes?search=term&page=1&limit=10&tag=work`
 - Search in `title` or `content`; filter by tag; return `{ data, page, total, pages }`.
5. **Security & DX:**
 - Error handling middleware; rate limit on `/auth` endpoints; `.env` config; CORS.

Frontend (React):

1. **Screens:** Login, Dashboard (list with search/pager), New/Edit Note modal, My Profile.
2. **Features:**
 - Create/Edit/Delete notes (owner-only).
 - Search bar, tag filter, pagination controls.
 - Snackbar/toast for success/fail; loading states.

Deliverables

- Working app (frontend + backend) with protected CRUD, search, pagination.
- README with setup, `.env.example`, and API docs.
- 2–3 screenshots + a 2–3 min demo video.

Sample Starter Code (Focused Snippets)

Validation & Pagination (Express):

js

```
// validate input (minimal example)
function validateNoteInput(req, res, next) {
  const { title, content } = req.body;
  if (!title || title.length < 3) return res.status(400).json({ message: "Title too short" });
  if (!content) return res.status(400).json({ message: "Content required" });
  next();
}

// list with search + pagination + tag
app.get("/api/notes", auth, async (req, res) => {
  const { page = 1, limit = 10, search = "", tag } = req.query;
  const q = {
    owner: req.user.id,
    ...(search && { $or: [{ title: { $regex: search, $options: "i" } },
      { content: { $regex: search, $options: "i" } } ] }),
    ...(tag && { tags: tag })
  };
  const skip = (Number(page) - 1) * Number(limit);
  const [data, total] = await Promise.all([
    Note.find(q).sort({ createdAt: -1 }).skip(skip).limit(Number(limit)),
    Note.countDocuments(q)
  ]);
  res.json({ data, page: Number(page), total, pages: Math.ceil(total / Number(limit)) });
});
```

Owner-Only Authorization (Express):

csharp

```
app.put("/api/notes/:id", auth, validateNote, async (req, res) => {
  const note = await Note.findById(req.params.id);
  if (!note) {
    return res.status(404).json({ message: "Not found" });
  }

  const isOwner = note.owner.toString() === req.user.id;
  const isAdmin = req.user.role === "admin";

  if (!isOwner && !isAdmin) {
    return res.status(403).json({ message: "Forbidden" });
  }

  note.title = req.body.title;
  note.content = req.body.content;
  note.tags = req.body.tags || [];

  await note.save();
  return res.json(note);
});
```

React: Search + Paging Controls (concept):

jsx

Copy

```
// state: search, page, tag
const [search, setSearch] = useState("");
const [page, setPage] = useState(1);
const [tag, setTag] = useState("");

useEffect(() => {
  api.get(`/notes?search=${encodeURIComponent(search)}&page=${page}&limit=10&tag=${tag}`);
}, [search, page, tag]);
```

🔴 Core Concepts (Required for Task-5)

1. Role-based authentication (User/Admin)
2. Authorization (ownership of note)
3. Search (MongoDB regex)
4. Pagination (skip/limit)
5. Validation (Joi or Zod or custom)
6. Better React structure & hooks

📺 Video Tutorials (Best Free)

Topic	Link
MERN Auth + Protected Routes	https://www.youtube.com/watch?v=7nafaH9SddU
MERN Notes Project Reference	https://www.youtube.com/watch?v=Z1Kt2N19lcQ
MongoDB Search (Regex)	https://www.youtube.com/watch?v=sVGowtrM9vI
Pagination Logic	https://www.youtube.com/watch?v=ZX3qt0UWifc

📖 Documentation + Articles

Concept	Link
MongoDB regex queries	https://www.mongodb.com/docs/manual/reference/operator/query/regex/
Pagination patterns	https://www.mongodb.com/docs/manual/reference/method/cursor.skip/
JWT usage	https://jwt.io/introduction/
Express validation	https://express-validator.github.io

🧰 Helpful Tools

- MongoDB Atlas (cloud DB)
- Postman (API testing)
- React DevTools

✅ Reference Starter Repositories

Repo	Purpose
MERN Notes Starter	https://github.com/bradtraversy/devconnector/tree/master
Simple MERN auth	https://github.com/abiduzz420/simple-mern-auth